

第4次实验 指令系统

1 实验目的

- (1) 掌握 MIPS、RISC-V、x86、ARMv7 汇编语言程序的编写与运行方法。
- (2) 熟悉 MARS、RARS、masm32、CPULator 以及 Logisim 仿真环境。
- (3) 理解不同指令系统的执行机制与程序控制方式。
- (4) 掌握排序程序在不同 ISA 下的实现方法。

2 实验环境

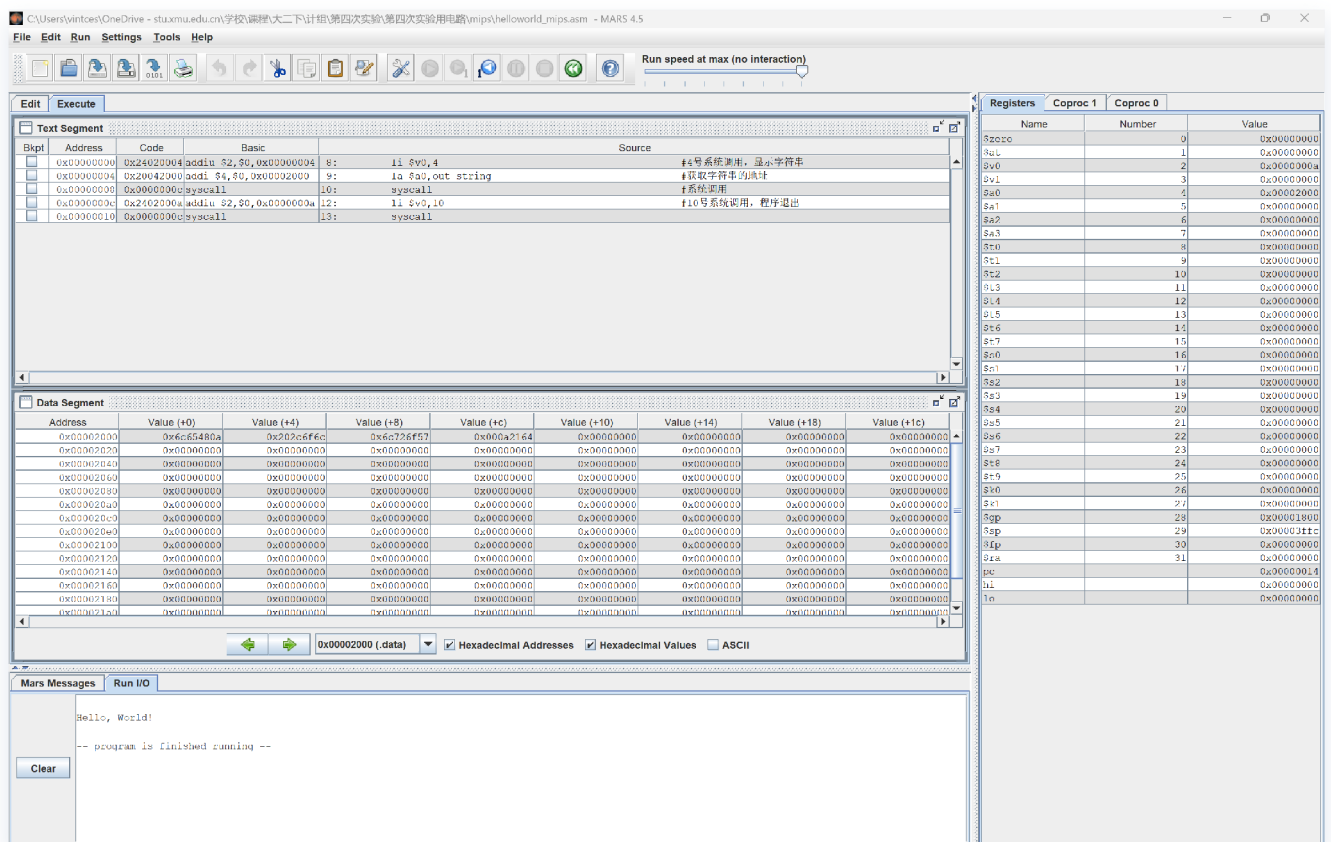
操作系统：Windows

软件工具：MARS 4.5、RARS 1.5、masm32、Logisim、CPULator

3.1 验证性实验

(1) MIPS 汇编语言程序的运行

截图



实验分析

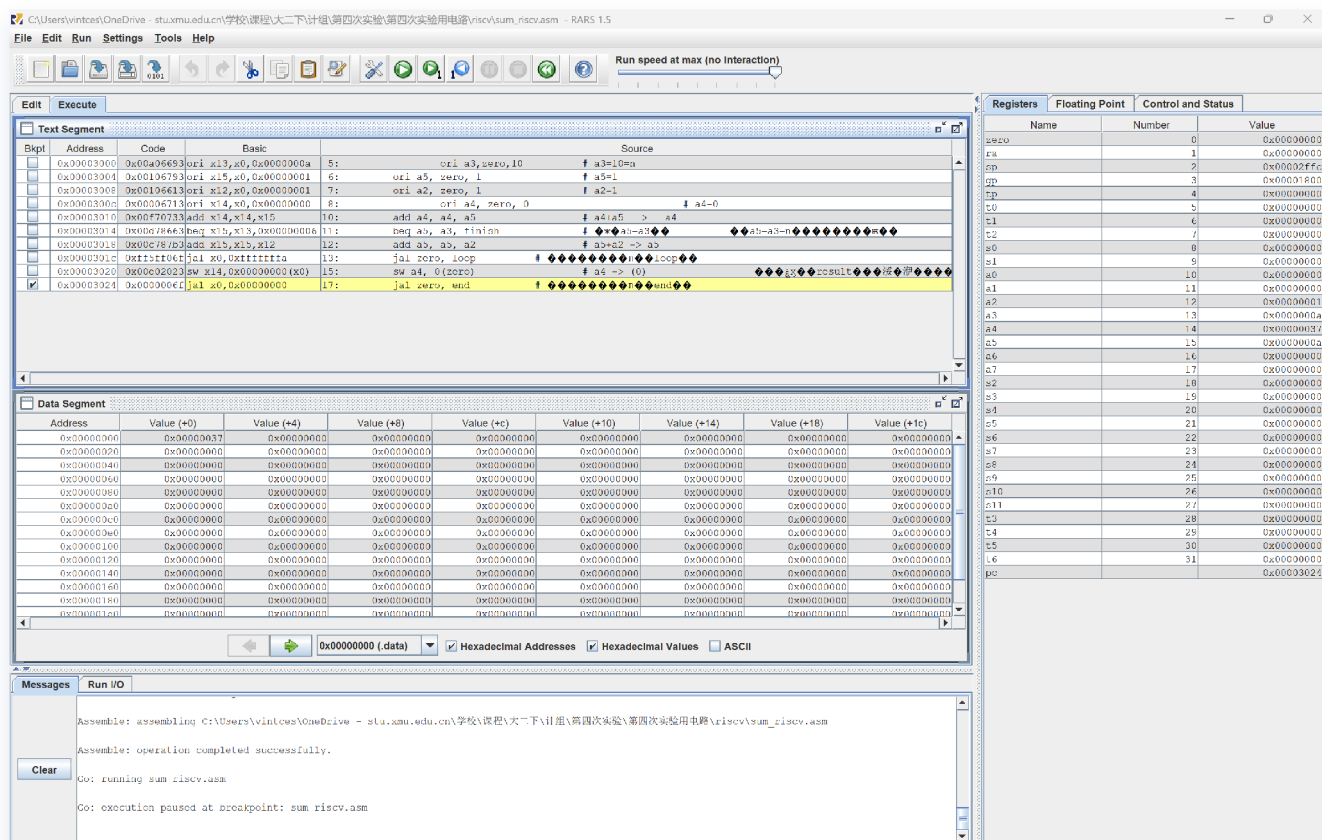
在 MARS 仿真环境中运行 Helloworld_mips.asm 程序，可以观察到程序被正确加载至 Text Segment，指令地址从 0x00000000 开始顺序排列。

程序执行后，在 Run I/O 窗口输出字符串 “Hello, World!”，说明程序成功调用系统调用实现字符串输出功能。同时，在数据段中可以观察到字符串对应的 ASCII 码存储于指定地址。

实验结果表明，MIPS 汇编程序能够正确完成字符串输出操作，验证了系统调用机制及程序执行流程的正确性。

(2)在Logisim上运行MIPS程序

截图

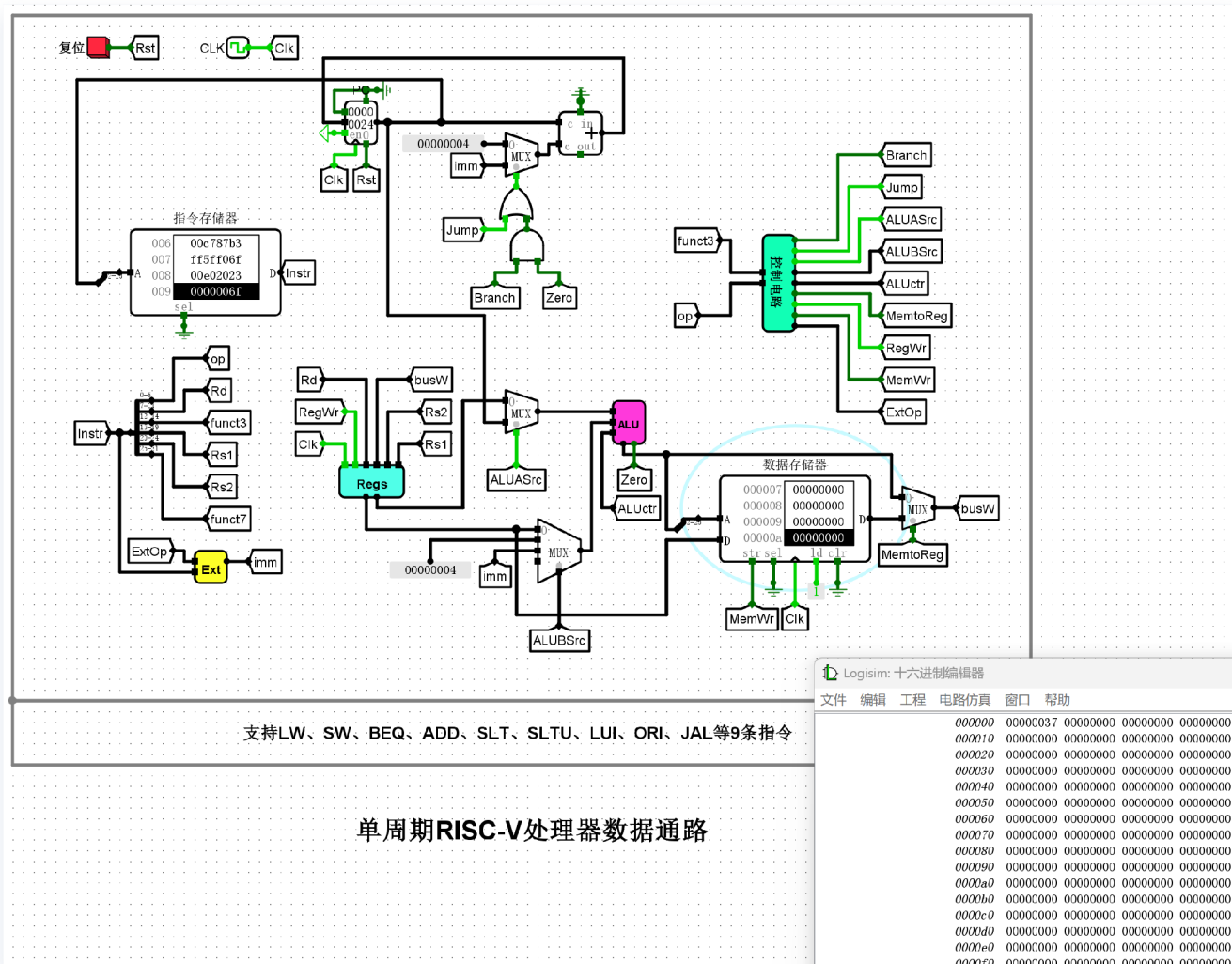


实验分析

在 RARS 1.5 中打开 `sum_riscv.asm`，将内存配置设为 Compact, Data at Address 0，汇编后在最后一行指令 `jal zero, end` 处设置断点并运行。程序执行完毕后停在断点处，右侧寄存器面板显示 `a4 = 0x00000037 = 55`，数据存储器 0 号单元（地址 `0x00000000`）的值为 `0x00000037`，即 $1+2+3+\dots+10 = 55$ ，结果正确，验证了 RISC-V 累加和程序运行成功。

(4)在Logisim上运行RISC-V程序

截图



实验分析

将 `sum_riscv.hex`（在文件头添加 `v2.0 raw`）装入 Logisim 实现的单周期 RISC-V 处理器的指令存储器，设置时钟频率为 1KHz，按总复位后启动时钟运行。待指令存储器内容停止滚动后停止时钟，程序执行完毕。观察数据存储器第 0 号单元（地址 `0x000000`）的值为 `0x00000037` = 55，即 $1+2+3+\dots+10 = 55$ ，结果正确，验证了 Logisim 实现的单周期 RISC-V 处理器能够正确运行该累加和程序。

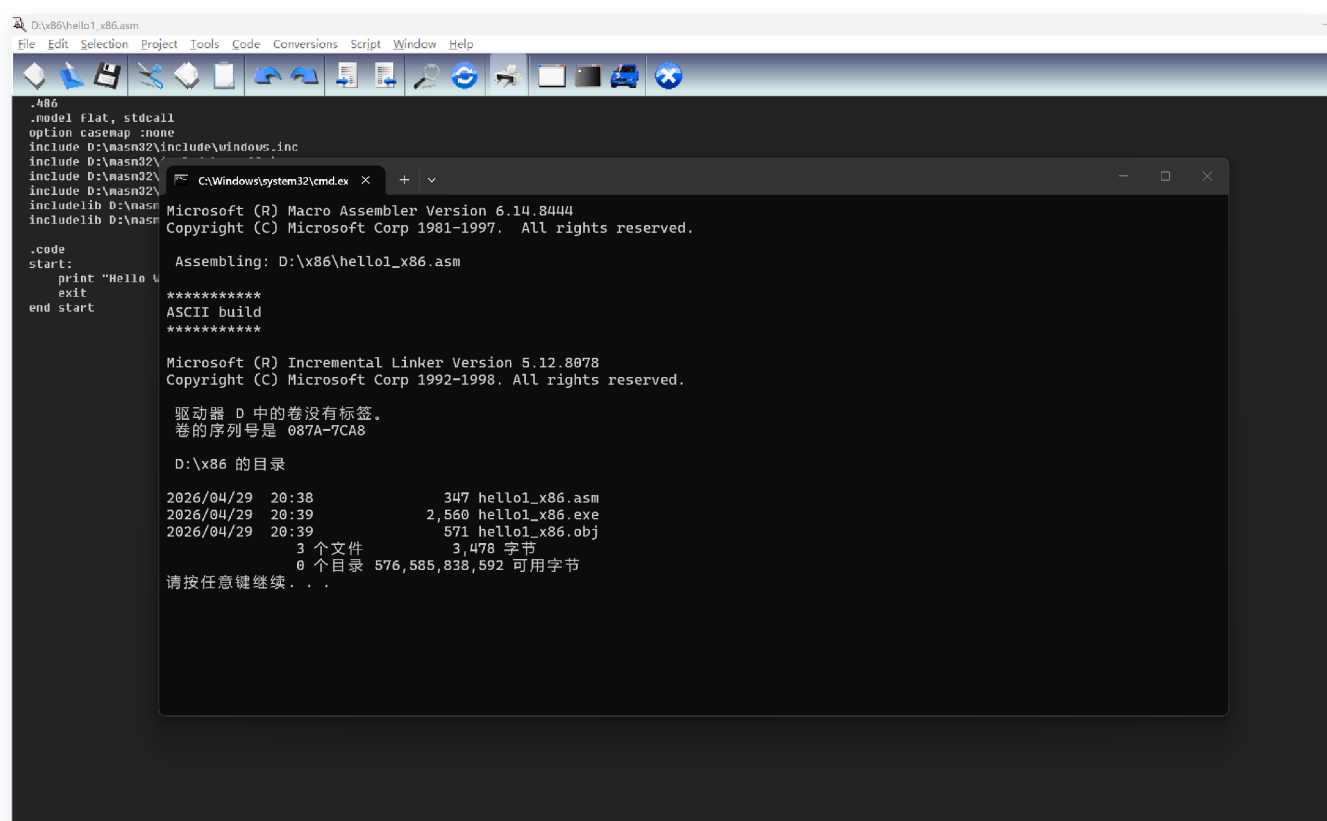
实验结果表明，RISC-V 程序能够在仿真环境中正确执行累加运算，同时验证了该指令系统在程序控制方式上的特点。

(5)Intel x86汇编语言程序的运行

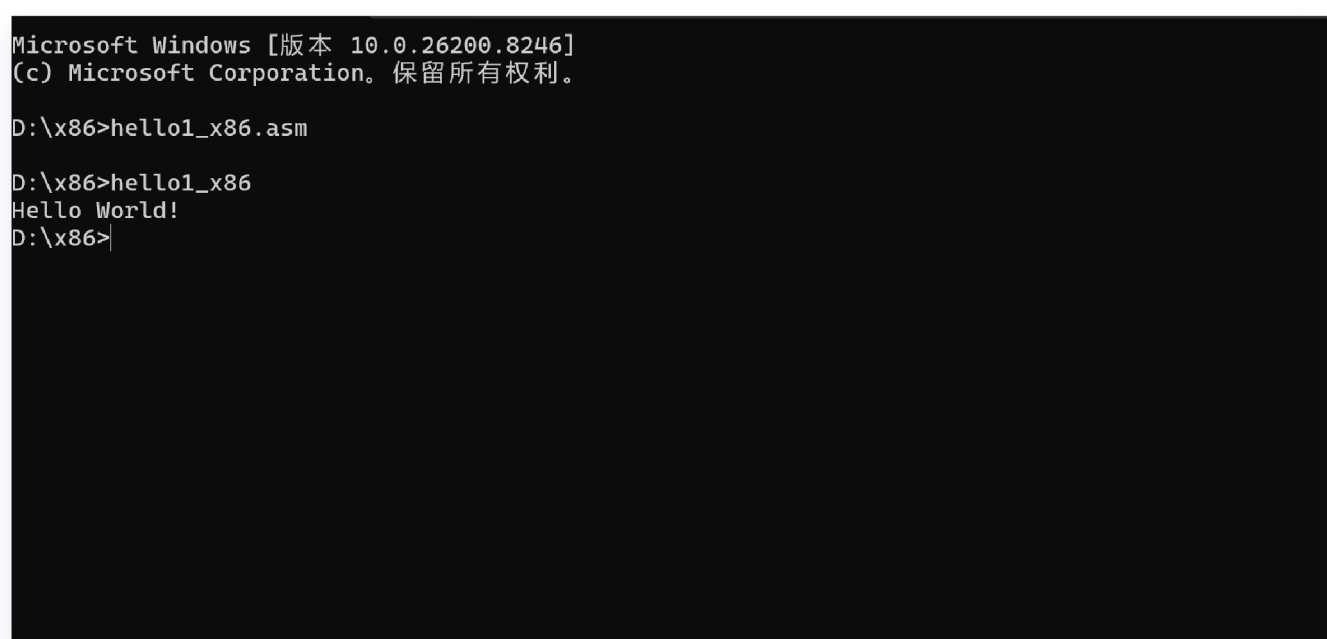
截图

`hello1_x86.asm`

编译完成截图



运行成功截图



sum_x86.asm

编译完成截图

```
Microsoft Windows [版本 10.0.26200.8246]  
(c) Microsoft Corporation。保留所有权利。  
  
D:\x86>hello1_x86.asm  
  
D:\x86>hello1_x86  
Hello World!  
D:\x86>
```

运行完成截图

```
Microsoft Windows [版本 10.0.26200.8246]  
(c) Microsoft Corporation。保留所有权利。  
  
D:\x86>sum_x86  
Input n= 10  
Sum= 55  
D:\x86>sum_x86  
Input n= 100  
Sum= 5050  
D:\x86>
```

实验分析

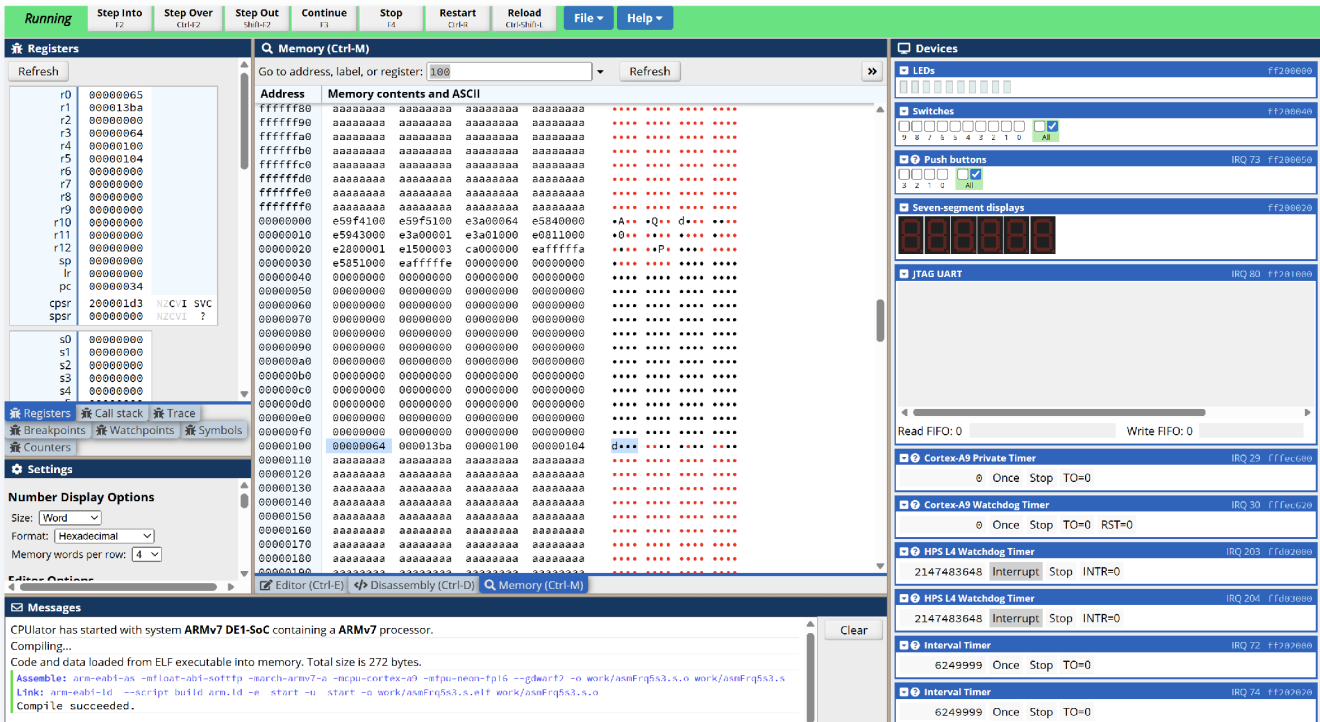
在 masm32 环境下，首先使用 qeditor 打开 hello1_x86.asm 程序，并通过汇编器（ml.exe）和连接器（link.exe）对程序进行编译和链接，生成可执行文件 hello1_x86.exe。运行该程序后，控制台输出“Hello World!”，说明程序能够正确调用输出宏，验证了 x86 汇编程序的基本运行流程。

随后，对 sum_x86.asm 程序进行编译和链接，生成 sum_x86.exe。运行程序并分别输入 n=10 和 n=100，得到输出结果分别为 55 和 5050，对应 $1+2+...+10=55$ 和 $1+2+...+100=5050$ ，结果正确。

该实验表明，在 x86 汇编语言中，通过循环结构实现累加运算，并结合输入输出宏完成数据交互，可以正确实现数值计算功能。同时也验证了 masm32 工具链（ml + link）在 Windows 环境下对 x86 汇编程序的编译与执行是正确可行的。

(6)ARMv7汇编语言程序的运行

截图



实验分析

在 CPULator 在线 ARMv7 仿真环境中加载并运行 sum_armv7.s 程序。程序运行后停在 stop 循环处，说明程序已经执行结束。观察寄存器可见 r1 的值为 0x000013ba，即十进制 5050；同时在 Memory 视图中，地址 0x104 处的值也为 0x000013ba。

程序中地址 0x100 处保存输入数据 0x00000064，即十进制 100；地址 0x104 处保存累加结果 0x000013ba，即 $1+2+3+...+100=5050$ 。运行结果正确，说明 ARMv7 汇编程序能够通过循环和条件跳转完成累加和运算。

回答问题

(1)Helloworld_mips.asm源程序中，前后两个syscall语句的各自作用是什么？

在本实验中，Helloworld_mips.asm 程序中包含两个 syscall 指令，其作用如下：

第一个 syscall（执行前设置 $v_0 = 4$ ， a_0 为字符串地址）用于调用系统提供的字符串输出服务，实现将指定字符串输出到控制台。

第二个 syscall（执行前设置 $v_0 = 10$ ）用于调用程序退出服务，使程序正常终止运行。

通过运行结果可以验证，这两个系统调用分别实现了输出功能和程序结束功能。

(2)修改sum_mips.asm源代码,设n=100, 修改后程序命名为sum100_mips.asm, 编译运行。要求展示所使用寄存器和内存的存储情况。若将其机器码导出，如何操作？导出文件命名为sum100_mips.hex。

在Logisim单周期MIPS处理器上运行sum100_mips.hex的结果：

数据存储器0号单元的值为 $0x000013ba = 5050$ ，
即 $1+2+3+\dots+100 = 5050$ ，结果正确。

程序运行统计：

- 总周期数：404
- 无条件分支数：99
- 条件分支成功数：1

分析：

循环体执行了100次（ $n=100$ ），每次循环包含 add、beq、add、j共4条指令，加上初始化4条和结束3条，总周期数为 $4 \times 100 + 7 = 407$ ，

与实测404略有差异是因为beq不成功的周期计入条件分支而非总循环。

无条件分支(j loop)执行了99次，最后一次通过beq成功跳转到finish，故条件分支成功数为1。

实验结果表明，修改后的程序能够正确完成累加运算，同时验证了寄存器与内存数据变化的正确性。

(3)在Logisim实现的单周期MIPS处理器上，运行计算 $1+2+3+\cdots+100$ 的机器语言程序（sum100_mips.hex），观看数据存储器0号单元的值是不是13bah？程序运行的总周期数、无条件分支数、条件分支成功数分别是多少？

在 Logisim 实现的单周期 MIPS 处理器上加载 sum100_mips.hex 程序并运行。程序执行结束后，观察数据存储器 0 号单元的值为 0x000013ba，即十进制 5050，说明 $1+2+3+\cdots+100$ 的计算结果正确。

程序运行统计如下：

- 总周期数：404
- 无条件分支数：99
- 条件分支成功数：1

分析：循环体执行 100 次，每次通过无条件跳转指令 j loop 返回循环起始位置，共执行 99 次；最后一次通过条件分支指令 beq 成功跳出循环，因此条件分支成功数为 1。总周期数与指令执行次数基本一致，验证了单周期处理器的执行特性。

(4)比较并分析sum_riscv.asm与sum_mips.asm两段代码在程序结束部分是如何处理的？

在 MIPS 汇编程序中，程序结束通常通过系统调用实现，例如：

```
li $v0, 10  
syscall
```

该方式通过操作系统提供的服务终止程序执行。

而在 RISC-V 汇编程序中，通常不依赖操作系统，而是采用无限循环的方式结束程序，例如：

```
jal zero, end
```

该指令不断跳转到自身，使程序停留在该位置，相当于“软停机”。

因此，两者的主要区别在于：

- MIPS：通过 syscall 系统调用结束程序（依赖操作系统）
- RISC-V：通过自循环方式结束程序（不依赖操作系统）

这反映了两种体系结构在程序控制方式上的差异。

(5) 修改 `sum_riscv.asm` 程序，计算 $1+2+3+\cdots+100$ （修改后的程序命名为：`sum100_riscv.asm`），然后在 RARS 1.5 汇编仿真器中运行，并将该程序的机器码保存到 `sum100_riscv.hex` 文件中。

将 `sum_riscv.asm` 程序中的初始值由 `n=10` 修改为 `n=100`（即将 `ori a3, zero, 10` 修改为 `ori a3, zero, 100`），保存为 `sum100_riscv.asm`。

在 RARS 1.5 中对该程序进行汇编和运行，程序执行结束后，观察寄存器和数据存储器中的结果为 `0x000013ba`，即十进制 5050，说明计算结果正确。

随后通过 `File → Dump Memory To File`，将机器码导出为十六进制文本格式文件，命名为 `sum100_riscv.hex`。

(6) 针对在 X86 ISA 架构下，提供的 5 个 `helloworld.asm` 程序段，分析比较它们在语法、调用关系，展示等环节有何不同？

在 x86 ISA 架构下，不同的 `helloworld.asm` 程序在语法、调用关系和输出方式上存在差异，主要体现在以下几个方面：

（1）语法差异

不同程序采用的汇编语法风格不同，如 MASM、NASM 等，其指令格式、段定义方式及伪指令存在差异。例如 MASM 使用 `.model`、`.code` 等结构，而 NASM 更接近裸汇编风格。

（2）调用关系不同

部分程序通过调用 Windows API（如 `WriteConsole` 或 `printf`）实现输出，而有些程序通过调用 C 运行库函数，甚至直接使用系统调用。这体现了 x86 程序可以运行在不同抽象层次之上。

（3）输出实现方式不同

有的程序使用宏（如 `print`）进行输出，隐藏了底层细节；有的程序需要手动压栈参数并调用函数；还有的程序通过中断或系统调用实现输出。

（4）程序结构差异

不同程序在入口定义（如 `start`、`main`）和结束方式（如 `exit`、`ret`）上有所不同，反映了不同开发环境和运行模式的差异。

综上，不同的 `helloworld` 程序展示了 x86 汇编语言在不同开发环境下的多样性和灵活性。

(7) 对于 `sum_x86.asm` 源程序，其中 `print` 语句的功能是怎样的？如果在 8086 ISA 架构下，其可替换为什么语句？请在编译器中验证，并给出相应的截屏佐证。

在 `sum_x86.asm` 源程序中，`print` 语句用于输出字符串或数值，其本质是一个宏定义，封装了底层的输出函数调用（通常是调用 Windows API 或 C 运行库函数）。

例如：

```
print "Hello World!"
```

该语句在汇编时会被展开为一系列调用输出函数的指令，从而实现字符串输出功能。

在 8086 ISA 架构下，没有类似的高级宏，可以使用中断方式实现输出，例如：

```
mov ah, 09h
mov dx, offset msg
int 21h
```

该方式通过 DOS 中断调用实现字符串输出功能。

通过实验验证，可以发现 `print` 宏简化了输出操作，而底层本质仍然是调用系统服务或中断机制完成输出。

(8)比较下fib_armv7.s与fib_riscv.asm、fib_mips.asm是如何完成输入数据和结果的存储的？

在 `fib_armv7.s`、`fib_riscv.asm` 和 `fib_mips.asm` 程序中，输入数据和结果的存储方式存在一定差异：

（1）输入方式

三种架构中，输入 `n` 通常通过寄存器初始化实现，例如直接将 `n` 赋值给某个寄存器，而不是通过用户输入。

（2）结果存储方式

三种程序均将计算得到的斐波那契数列依次存储到数据存储器中：

- MIPS：通过 `sw` 指令将结果存入内存
- RISC-V：通过 `sw` 指令实现相同功能
- ARMv7：通过 `str` 指令将结果写入内存

（3）存储地址形式

三者均采用连续内存地址存储结果，即地址 0、4、8、12...依次存放数列元素。

总体来看，三种架构在实现逻辑上类似，仅在指令格式和寄存器命名上存在差异。

(9)fib_armv7.s中为何有数据段.word，是否能改成与其他两套指令代码一样的形式？

```
.org 0x100
num:
    .word 10
.org 0x200
result:
    .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

在 fib_armv7.s 程序中，使用了数据段定义，例如：

```
.org 0x100
num:
    .word 10

.org 0x200
result:
    .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```

其中，.word 指令用于在内存中分配一个或多个 32 位存储单元，并对其进行初始化。例如：

- num 用于存储输入值 n
- result 用于存储计算得到的斐波那契数列结果

这种方式属于“显式数据段定义”，其优点是：

- 明确指定数据的存储地址（如 0x100、0x200）
- 程序结构清晰，数据与代码分离
- 便于在仿真器中直接观察数据变化

相比之下，MIPS 和 RISC-V 程序中通常没有显式定义数据段，而是通过寄存器初始化数据，并在运行过程中通过 sw 指令将结果写入内存，因此代码形式上看起来更简洁。

理论上，可以将 ARMv7 程序改写为类似 MIPS 或 RISC-V 的形式，即：

- 不使用 .word 定义数据
- 直接通过寄存器设置初值
- 在程序运行过程中动态写入内存

但这样做会降低程序的可读性和结构性，并且不利于调试和观察数据。

因此，ARMv7 中使用 .word 定义数据是一种更规范、更清晰的做法，尤其适用于需要明确数据存储位置的程序设计。

3.2 设计性实验

(1)MIPS汇编语言程序设计（排序程序）

sort1_mips.asm

代码

```
# sort1_mips.asm
# MIPS降序排序程序
# 原始数据: 8,1,5,2,7,9,6,4,3,10
# 排序结果: 10,9,8,7,6,5,4,3,2,1

main:
    # 初始化数据到数据存储器 0,4,8, ..., 36
    addi $t0, $zero, 8
    sw    $t0, 0($zero)
    addi $t0, $zero, 1
    sw    $t0, 4($zero)
    addi $t0, $zero, 5
    sw    $t0, 8($zero)
    addi $t0, $zero, 2
    sw    $t0, 12($zero)
    addi $t0, $zero, 7
    sw    $t0, 16($zero)
    addi $t0, $zero, 9
    sw    $t0, 20($zero)
    addi $t0, $zero, 6
    sw    $t0, 24($zero)
    addi $t0, $zero, 4
    sw    $t0, 28($zero)
    addi $t0, $zero, 3
    sw    $t0, 32($zero)
    addi $t0, $zero, 10
    sw    $t0, 36($zero)

    # 冒泡排序: 降序
    addi $s0, $zero, 0      # i = 0
    addi $s2, $zero, 9      # 外层循环次数 9
```

```

outer_loop:
    slt  $t0, $s0, $s2      # i < 9 ?
    beq  $t0, $zero, finish

    addi $s1, $zero, 0      # j = 0
    sub  $s3, $s2, $s0      # 内层比较次数 = 9 - i

inner_loop:
    slt  $t0, $s1, $s3      # j < 9-i ?
    beq  $t0, $zero, next_outer

    sll  $t1, $s1, 2        # t1 = j * 4
    lw   $t2, 0($t1)        # 当前元素
    lw   $t3, 4($t1)        # 后一个元素

    # 降序: 如果 当前元素 < 后一个元素, 则交换
    slt  $t4, $t2, $t3
    beq  $t4, $zero, no_swap

    sw   $t3, 0($t1)
    sw   $t2, 4($t1)

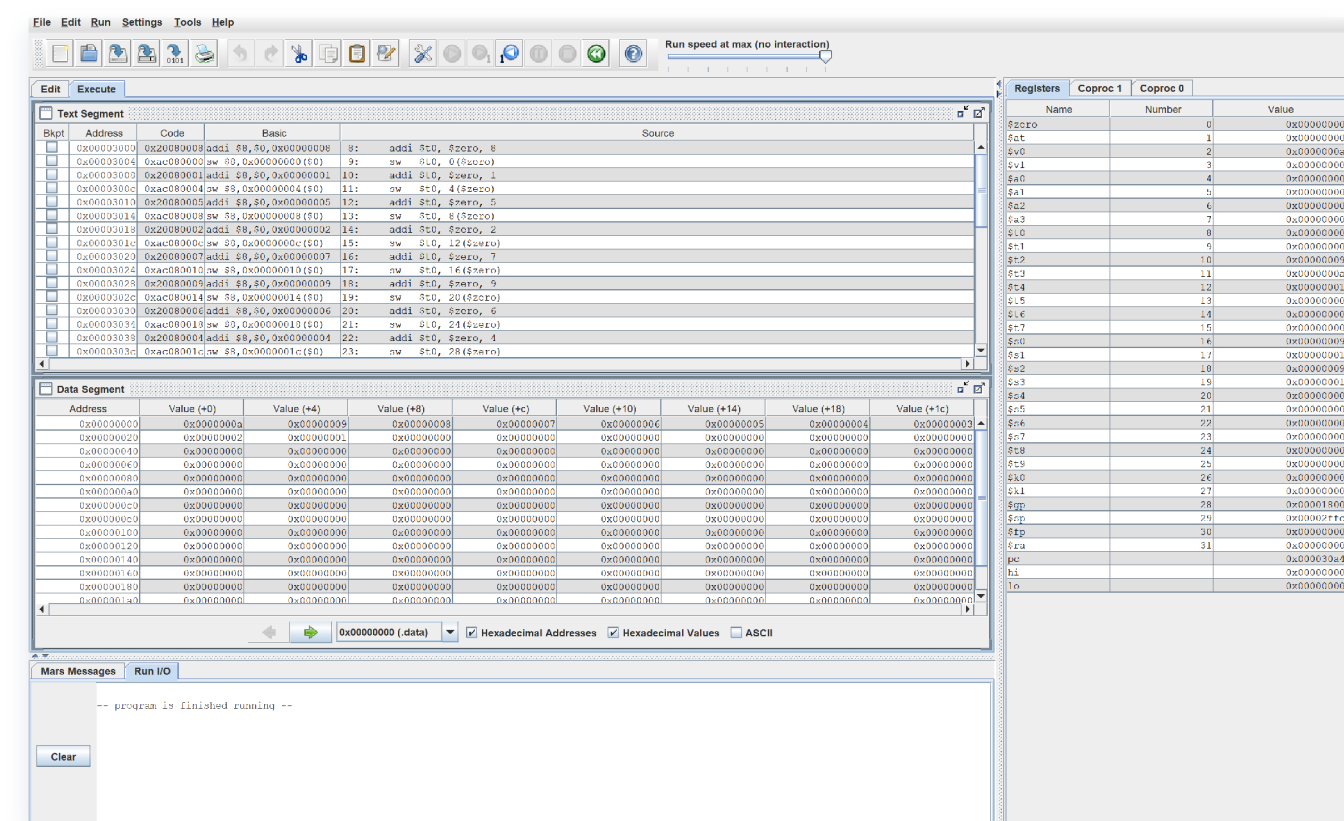
no_swap:
    addi $s1, $s1, 1
    j    inner_loop

next_outer:
    addi $s0, $s0, 1
    j    outer_loop

finish:
    addi $v0, $zero, 10
    syscall

```

运行截图



实验分析

本实验采用 MIPS 汇编语言实现了一个降序排序程序（sort1_mips.asm），对 10 个整数数据进行排序。程序首先将初始数据（8,1,5,2,7,9,6,4,3,10）依次存入数据存储器地址 0、4、8、...、36 中。

排序部分采用冒泡排序算法，通过两层循环实现数据比较与交换：外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素小于后一个元素时，进行交换，从而实现降序排列。

程序运行结束后，观察数据存储器中的内容，地址 0~36 处的数据依次为：

10, 9, 8, 7, 6, 5, 4, 3, 2, 1

与预期的降序结果一致，说明排序程序运行正确。

实验表明，利用 MIPS 指令（如 lw、sw、slt、beq、j 等）可以实现基本的排序算法逻辑，并验证了单周期处理器在处理循环和条件分支指令时的正确性。

sort2_mips.asm

代码

```
# sort2_mips.asm
# MIPS升序排序程序
# 原始数据: 8,1,5,2,7,9,6,4,3,10
```

排序结果: 1,2,3,4,5,6,7,8,9,10

main:

初始化数据到数据存储器 0,4,8, ..., 36

```
addi $t0, $zero, 8
sw    $t0, 0($zero)
addi $t0, $zero, 1
sw    $t0, 4($zero)
addi $t0, $zero, 5
sw    $t0, 8($zero)
addi $t0, $zero, 2
sw    $t0, 12($zero)
addi $t0, $zero, 7
sw    $t0, 16($zero)
addi $t0, $zero, 9
sw    $t0, 20($zero)
addi $t0, $zero, 6
sw    $t0, 24($zero)
addi $t0, $zero, 4
sw    $t0, 28($zero)
addi $t0, $zero, 3
sw    $t0, 32($zero)
addi $t0, $zero, 10
sw    $t0, 36($zero)
```

冒泡排序: 升序

```
addi $s0, $zero, 0      # i = 0
addi $s2, $zero, 9      # 外层循环次数 9
```

outer_loop:

```
slt  $t0, $s0, $s2      # i < 9 ?
beq  $t0, $zero, finish

addi $s1, $zero, 0      # j = 0
sub  $s3, $s2, $s0      # 内层比较次数 = 9 - i
```

inner_loop:

```
slt  $t0, $s1, $s3      # j < 9-i ?
beq  $t0, $zero, next_outer

sll  $t1, $s1, 2         # t1 = j * 4
lw   $t2, 0($t1)         # 当前元素
lw   $t3, 4($t1)         # 后一个元素
```

```
slt    $t4, $t3, $t2
beq    $t4, $zero, no_swap
```

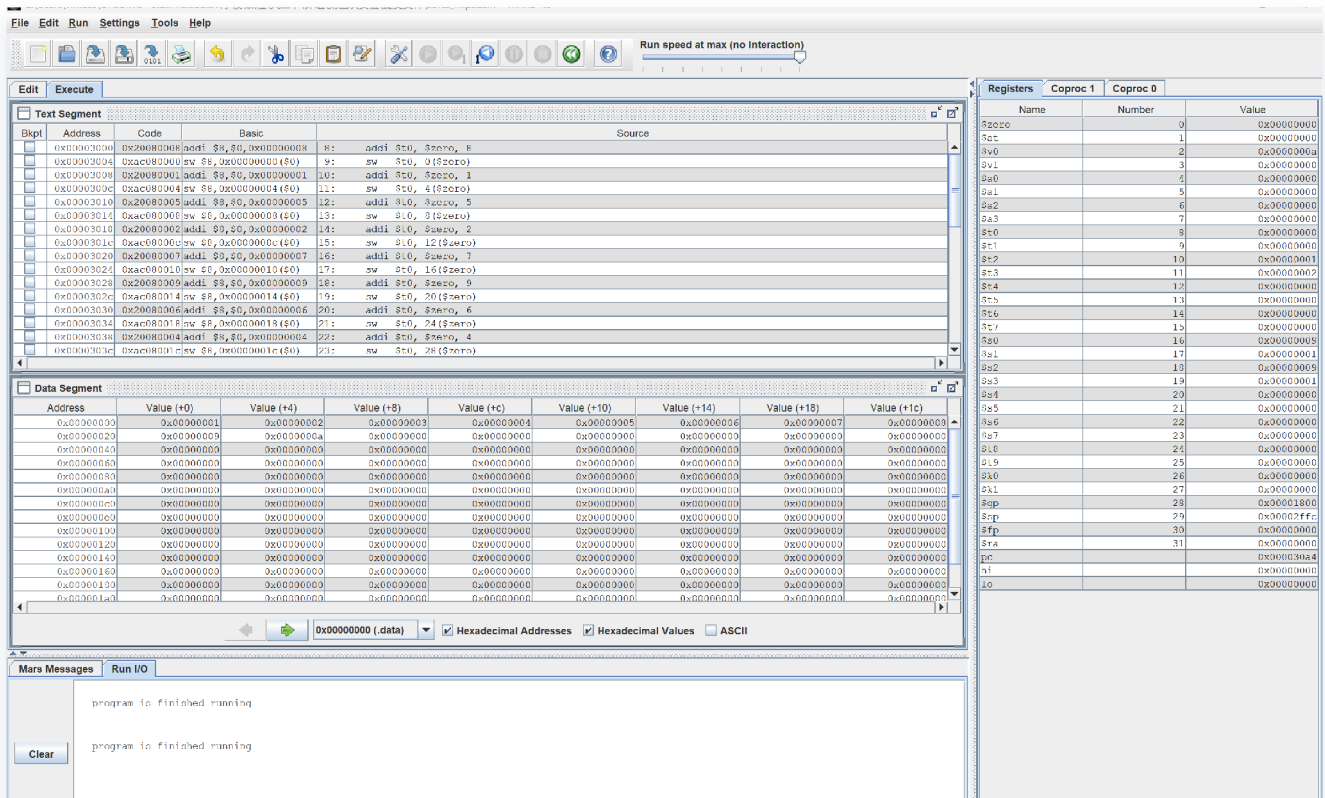
SW \$t3, 0(\$t1)
SW \$t2, 4(\$t1)

```
no_swap:
    addi $s1, $s1, 1
    j     inner_loop
```

```
next_outer:
    addi $s0, $s0, 1
    j    outer_loop
```

```
finish:
    addi $v0, $zero, 10
    syscall
```

运行截图



实验分析

本实验在 MIPS 汇编语言中实现了升序排序程序（sort2_mips.asm）。程序首先将 10 个待排序数据存入数据存储器地址 0、4、8、...、36 中，然后采用冒泡排序算法进行排序。

排序过程中，程序通过两层循环依次比较相邻元素。当当前元素大于后一个元素时，交换二者的位置，使较小的数据逐渐移动到前面。程序主要使用 lw、sw、slt、beq、j 等指令完成数据读取、比较、交换和循环控制。

程序运行结束后，观察数据存储器中的内容，地址 0~36 处的数据依次为：

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

结果与预期升序排列一致，说明 sort2_mips.asm 程序运行正确。

(2)RISC-V汇编语言程序设计（排序程序）

sort1_riscv.asm

代码

```
# sort1_riscv.asm
# RISC-V 降序排序
# 原始数据: 8,1,5,2,7,9,6,4,3,10
# 排序结果: 10,9,8,7,6,5,4,3,2,1

main:
    # 初始化数据
    ori t0, zero, 8
    sw t0, 0(zero)
    ori t0, zero, 1
    sw t0, 4(zero)
    ori t0, zero, 5
    sw t0, 8(zero)
    ori t0, zero, 2
    sw t0, 12(zero)
    ori t0, zero, 7
    sw t0, 16(zero)
    ori t0, zero, 9
    sw t0, 20(zero)
    ori t0, zero, 6
    sw t0, 24(zero)
    ori t0, zero, 4
    sw t0, 28(zero)
    ori t0, zero, 3
```

```

sw t0, 32(zero)
ori t0, zero, 10
sw t0, 36(zero)

ori s0, zero, 0      # i = 0
ori s2, zero, 9      # 外层循环次数

```

outer_loop:

```

slt t0, s0, s2      # i < 9 ?
beq t0, zero, finish

ori s1, zero, 0      # j = 0
sub s3, s2, s0       # 内层次数 = 9 - i

```

inner_loop:

```

slt t0, s1, s3      # j < 9-i ?
beq t0, zero, next_outer

slli t1, s1, 2       # t1 = j * 4
lw t2, 0(t1)         # 当前元素
lw t3, 4(t1)         # 后一个元素

```

降序: 当前元素 < 后一个元素, 则交换

```

slt t4, t2, t3
beq t4, zero, no_swap

```

```

sw t3, 0(t1)
sw t2, 4(t1)

```

no_swap:

```

addi s1, s1, 1
jal zero, inner_loop

```

next_outer:

```

addi s0, s0, 1
jal zero, outer_loop

```

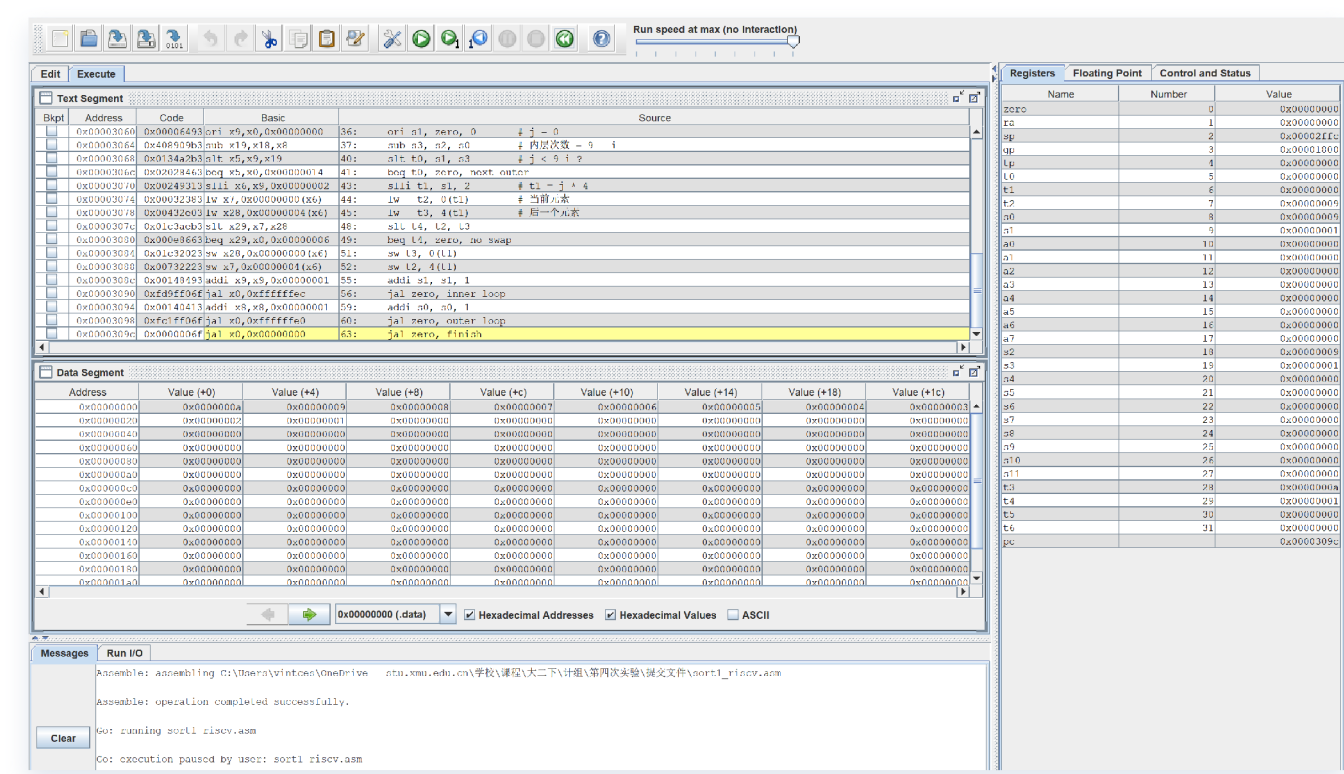
finish:

```

jal zero, finish

```

运行截图



实验分析

本实验使用 RISC-V 汇编语言实现了降序排序程序（sort1_riscv.asm）。程序首先将 10 个待排序数据（8,1,5,2,7,9,6,4,3,10）依次存入数据存储器地址 0、4、8、...、36 中。

排序过程采用冒泡排序算法，通过两层循环实现数据的比较与交换。外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素小于后一个元素时，交换两者位置，从而实现降序排列。

程序运行结束后，观察数据存储器中的内容，地址 0~36 处的数据依次为：
10, 9, 8, 7, 6, 5, 4, 3, 2, 1

与预期结果一致，说明排序程序运行正确。

实验表明，RISC-V 汇编语言可以通过基本指令（如 lw、sw、slt、beq、jal 等）实现循环控制与条件判断，从而完成排序算法。同时也验证了 RARS 仿真环境对 RISC-V 程序执行的正确性。

sort2_riscv.asm

代码

```
# sort2_riscv.asm
# RISC-V 升序排序
# 原始数据: 8,1,5,2,7,9,6,4,3,10
# 排序结果: 1,2,3,4,5,6,7,8,9,10
```

main:

初始化数据

```
ori t0, zero, 8
sw t0, 0(zero)
ori t0, zero, 1
sw t0, 4(zero)
ori t0, zero, 5
sw t0, 8(zero)
ori t0, zero, 2
sw t0, 12(zero)
ori t0, zero, 7
sw t0, 16(zero)
ori t0, zero, 9
sw t0, 20(zero)
ori t0, zero, 6
sw t0, 24(zero)
ori t0, zero, 4
sw t0, 28(zero)
ori t0, zero, 3
sw t0, 32(zero)
ori t0, zero, 10
sw t0, 36(zero)
```

```
ori s0, zero, 0      # i = 0
ori s2, zero, 9      # 外层循环次数
```

outer_loop:

```
slt t0, s0, s2      # i < 9 ?
beq t0, zero, finish

ori s1, zero, 0      # j = 0
sub s3, s2, s0       # 内层次数 = 9 - i
```

inner_loop:

```
slt t0, s1, s3      # j < 9-i ?
beq t0, zero, next_outer

slli t1, s1, 2       # t1 = j * 4
lw t2, 0(t1)         # 当前元素
lw t3, 4(t1)         # 后一个元素

# 升序: 当前元素 > 后一个元素, 则交换
slt t4, t3, t2
beq t4, zero, no_swap
```

```
sw t3, 0(t1)
sw t2, 4(t1)
```

no_swap:

```
addi s1, s1, 1
jal zero, inner_loop
```

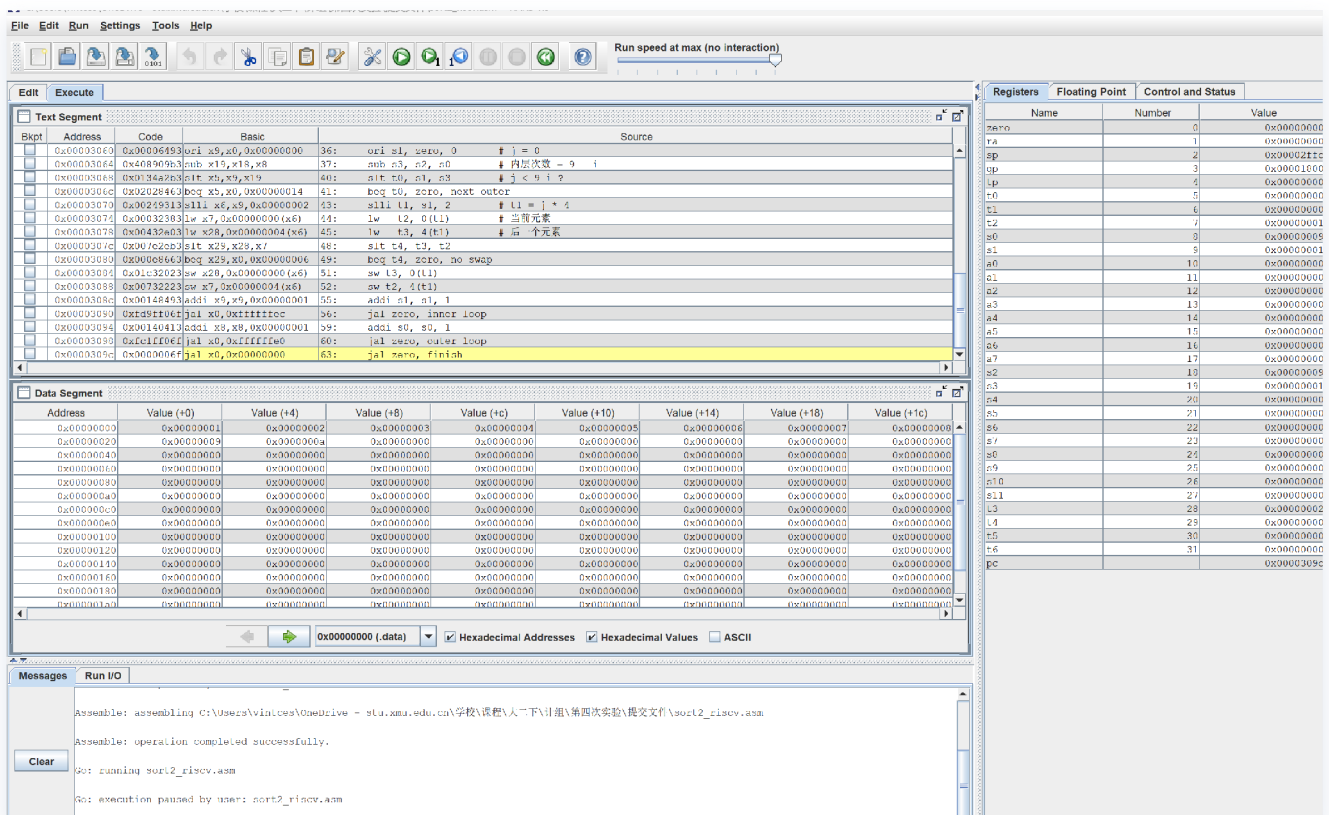
next_outer:

```
addi s0, s0, 1
jal zero, outer_loop
```

finish:

```
jal zero, finish
```

运行截图



实验分析

本实验使用 RISC-V 汇编语言实现了升序排序程序（sort2_riscv.asm）。程序首先将 10 个待排序数据（8,1,5,2,7,9,6,4,3,10）依次存入数据存储器地址 0、4、8、...、36 中。

排序过程采用冒泡排序算法，通过两层循环实现数据比较与交换。外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素大于后一个元素时，交换两者位置，使较小的数据逐渐移动到前面，从而实现升序排列。

程序运行结束后，观察数据存储器中的内容，地址 0~36 处的数据依次为：

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

结果与预期升序排列一致，说明排序程序运行正确。

实验表明，RISC-V 汇编语言能够通过基本指令（如 lw、sw、slt、beq、jal 等）实现循环结构和条件判断，从而完成排序算法。同时验证了 RARS 仿真环境对程序执行结果的正确性。

(3) Intel x86 汇编语言程序设计（排序程序）

sort1_x86.asm

代码

```
.486
.model flat, stdcall
option casemap :none

include D:\masm32\include\windows.inc
include D:\masm32\include\masm32.inc
include D:\masm32\include\kernel32.inc
include D:\masm32\macros\macros.asm

includelib D:\masm32\lib\masm32.lib
includelib D:\masm32\lib\kernel32.lib

.data
array dd 8, 1, 5, 2, 7, 9, 6, 4, 3, 10

.code
start:

    ; 冒泡排序，降序
    mov ecx, 9                ; 外层循环次数

outer_loop:
    push ecx
    mov esi, 0                ; 数组下标偏移
    mov ecx, 9                ; 内层比较次数

inner_loop:
    mov eax, array[esi]
    mov ebx, array[esi+4]
```

; 降序: 如果前一个 < 后一个, 则交换

cmp eax, ebx

jge no_swap

mov array[esi], ebx

mov array[esi+4], eax

no_swap:

add esi, 4

loop inner_loop

pop ecx

loop outer_loop

; 输出排序结果

print chr\$("Sorted result descending:", 13, 10)

mov ecx, 10

mov esi, 0

print_loop:

push ecx

mov eax, array[esi]

print str\$(eax)

print chr\$(" ")

add esi, 4

pop ecx

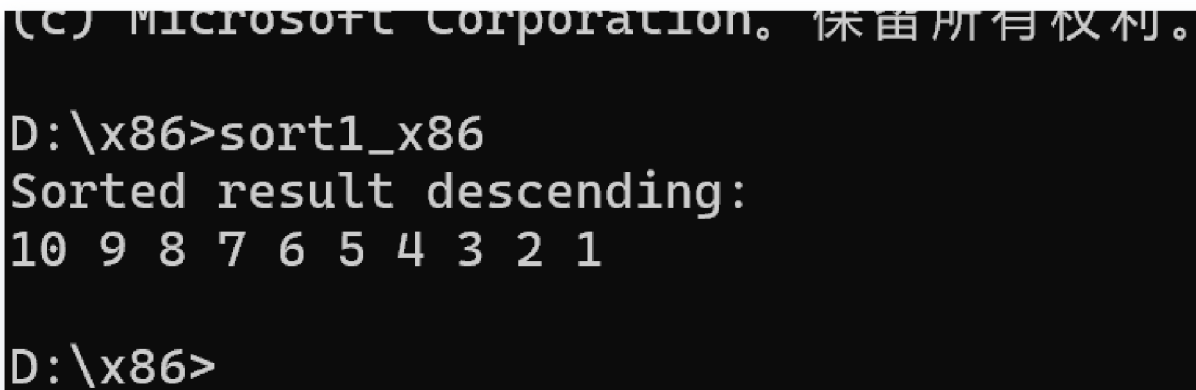
loop print_loop

print chr\$(13, 10)

exit

end start

运行截图



```
(C) Microsoft Corporation。保留所有权利。  
  
D:\x86>sort1_x86  
Sorted result descending:  
10 9 8 7 6 5 4 3 2 1  
  
D:\x86>
```

实验分析

本实验使用 Intel x86 汇编语言实现了降序排序程序（sort1_x86.asm）。程序首先在数据段中定义了一个包含 10 个整数的数组（8,1,5,2,7,9,6,4,3,10），然后采用冒泡排序算法对其进行排序。

排序过程中，通过两层循环控制比较和交换操作。外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素小于后一个元素时，交换二者位置，从而实现降序排列。程序主要使用寄存器（如 eax、ebx、ecx、esi）以及 cmp、jge、loop 等指令完成排序逻辑。

排序完成后，通过 print 宏逐个输出数组元素，最终结果为：

10 9 8 7 6 5 4 3 2 1

与预期降序结果一致，说明程序运行正确。该实验验证了 x86 汇编语言在 Windows 环境下通过 MASM32 工具链可以实现基本排序算法，同时体现了寄存器操作与循环控制在程序设计中的作用。

sort2_x86.asm

代码

```
.486  
.model flat, stdcall  
option casemap :none  
  
include D:\masm32\include\windows.inc  
include D:\masm32\include\masm32.inc  
include D:\masm32\include\kernel32.inc  
include D:\masm32\macros\macros.asm  
  
includelib D:\masm32\lib\masm32.lib
```

```

includelib D:\masm32\lib\kernel32.lib

.data
array dd 8, 1, 5, 2, 7, 9, 6, 4, 3, 10

.code
start:

    ; 冒泡排序, 升序
    mov ecx, 9                ; 外层循环次数

outer_loop:
    push ecx
    mov esi, 0                ; 数组下标偏移
    mov ecx, 9                ; 内层比较次数

inner_loop:
    mov eax, array[esi]
    mov ebx, array[esi+4]

    ; 升序: 如果前一个 > 后一个, 则交换
    cmp eax, ebx
    jle no_swap

    mov array[esi], ebx
    mov array[esi+4], eax

no_swap:
    add esi, 4
    loop inner_loop

    pop ecx
    loop outer_loop

    ; 输出排序结果
    print chr$("Sorted result ascending:", 13, 10)

    mov ecx, 10
    mov esi, 0

print_loop:
    push ecx

    mov eax, array[esi]

```

```

    print str$(eax)
    print chr$(" ")

    add esi, 4
    pop ecx
    loop print_loop

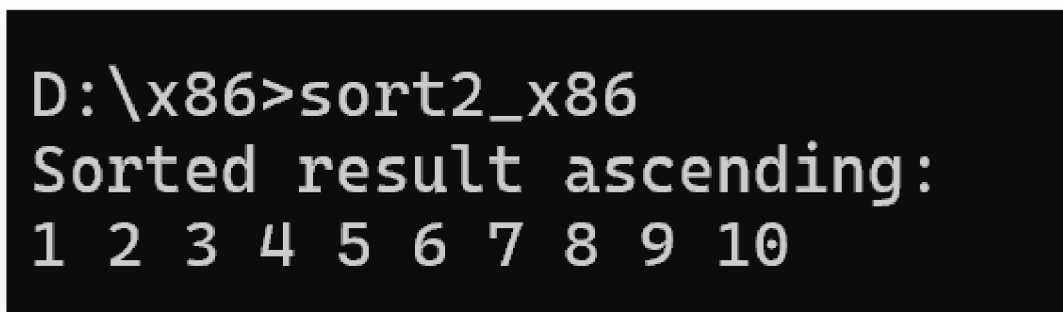
    print chr$(13, 10)

    exit

end start

```

运行截图



```

D:\x86>sort2_x86
Sorted result ascending:
1 2 3 4 5 6 7 8 9 10

```

实验分析

本实验使用 Intel x86 汇编语言实现了升序排序程序（sort2_x86.asm）。程序在数据段中定义了一个包含 10 个整数的数组（8,1,5,2,7,9,6,4,3,10），并采用冒泡排序算法对其进行排序。

排序过程中，通过两层循环控制比较与交换操作。外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素大于后一个元素时，交换两者位置，使较小的元素逐步移动到数组前端，从而实现升序排列。

程序使用 eax、ebx、ecx、esi 等寄存器进行数据操作，并结合 cmp、jle、loop 等指令实现条件判断与循环控制。排序完成后，通过 print 宏依次输出数组元素，最终结果为：

1 2 3 4 5 6 7 8 9 10

结果与预期升序排列一致，说明程序运行正确。实验验证了 x86 汇编语言在 MASM32 环境下可以实现基本排序算法，并体现了寄存器操作和控制流指令在程序设计中的作用。

3.3 挑战性实验

ARMv7汇编语言程序设计（排序程序）

sort1_armv7.s

代码

```
.global _start

_start:
    ldr r4, =array    // r4 = 数组首地址
    mov r0, #9        // 外层循环次数 i = 9

outer_loop:
    cmp r0, #0
    beq stop

    mov r1, #0        // j = 0
    mov r2, r0        // 内层比较次数

inner_loop:
    cmp r1, r2
    beq next_outer

    lsl r3, r1, #2     // r3 = j * 4
    add r5, r4, r3     // r5 = array[j] 地址

    ldr r6, [r5]       // r6 = array[j]
    ldr r7, [r5, #4]   // r7 = array[j+1]

    // 降序: 如果 array[j] < array[j+1], 交换
    cmp r6, r7
    bge no_swap

    str r7, [r5]
    str r6, [r5, #4]

no_swap:
    add r1, r1, #1
    b inner_loop

next_outer:
    sub r0, r0, #1
    b outer_loop
```

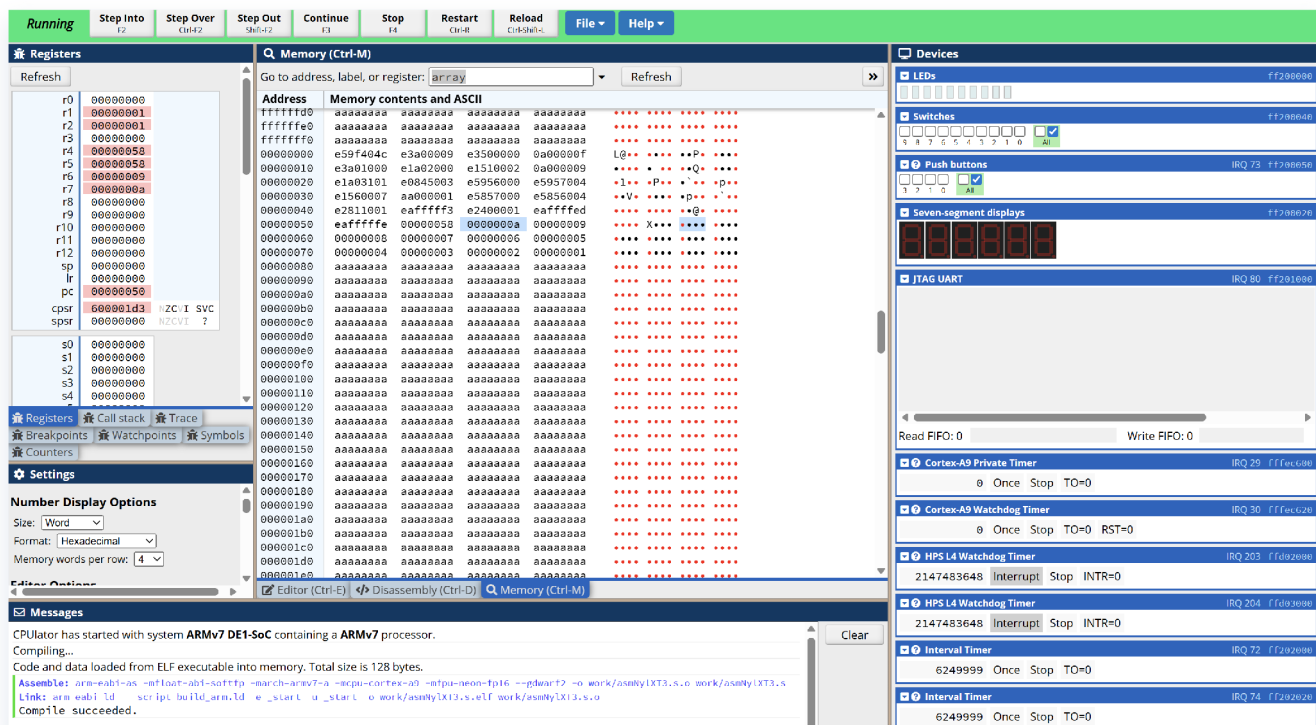
```

stop:
    b stop

.data
array:
    .word 8, 1, 5, 2, 7, 9, 6, 4, 3, 10

```

运行截图



实验分析

本实验使用 ARMv7 汇编语言实现了降序排序程序（sort1_armv7.s）。程序在数据段中定义了一个包含 10 个整数的数组（8,1,5,2,7,9,6,4,3,10），并通过寄存器 r4 保存数组首地址。

排序过程采用冒泡排序算法，通过两层循环实现相邻元素的比较与交换。外层循环控制排序趟数，内层循环逐对比较数组元素。当当前元素小于后一个元素时，使用 str 指令交换两者位置，从而实现降序排列。

程序运行结束后，在 Memory 视图中通过标签 array 定位数组起始地址，可以看到数组内容依次为：10, 9, 8, 7, 6, 5, 4, 3, 2, 1

结果与预期降序排列一致，说明 sort1_armv7.s 程序运行正确。实验表明，ARMv7 汇编语言可以通过 ldr、str、cmp、bge、b 等指令实现排序算法中的数据访问、条件判断和循环控制。

sort2_armv7.s

代码

```
.global _start

_start:
    ldr r4, =array    // r4 = 数组首地址
    mov r0, #9        // 外层循环次数 i = 9

outer_loop:
    cmp r0, #0
    beq stop

    mov r1, #0        // j = 0
    mov r2, r0        // 内层比较次数

inner_loop:
    cmp r1, r2
    beq next_outer

    lsl r3, r1, #2     // r3 = j * 4
    add r5, r4, r3     // r5 = array[j] 地址

    ldr r6, [r5]       // r6 = array[j]
    ldr r7, [r5, #4]   // r7 = array[j+1]

    // 升序: 如果 array[j] > array[j+1], 交换
    cmp r6, r7
    ble no_swap

    str r7, [r5]
    str r6, [r5, #4]

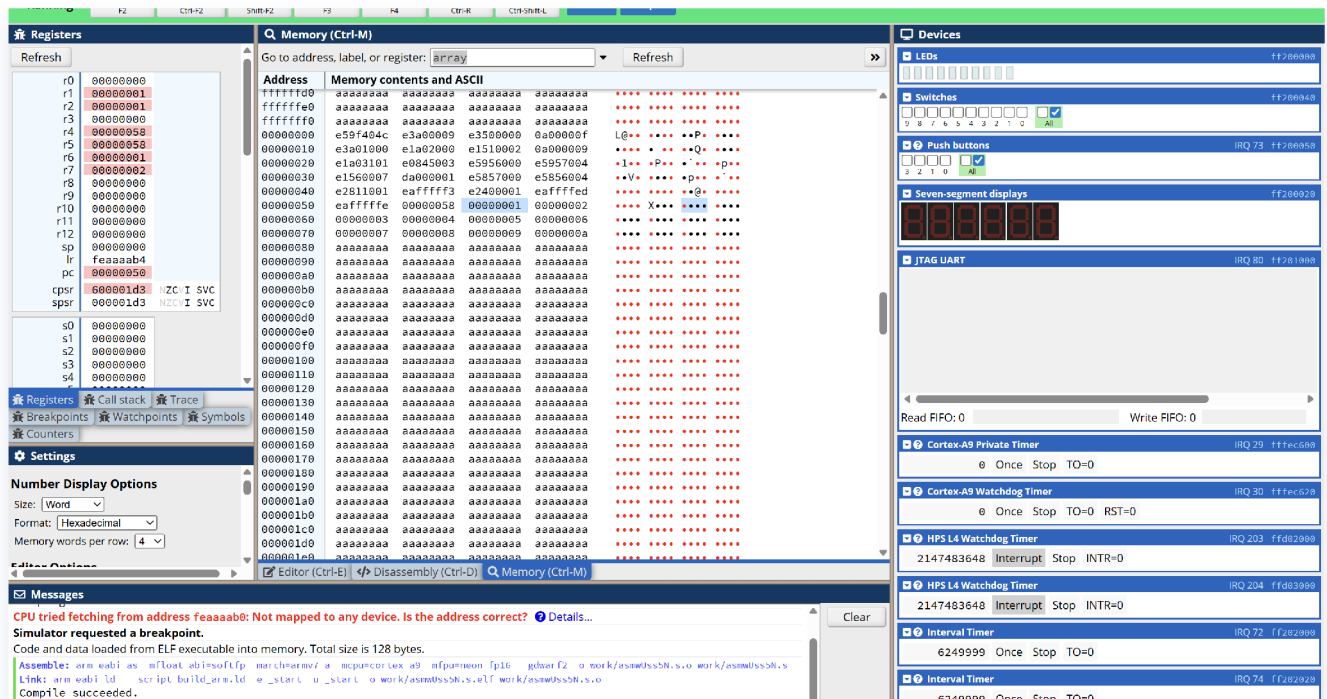
no_swap:
    add r1, r1, #1
    b inner_loop

next_outer:
    sub r0, r0, #1
    b outer_loop

stop:
    b stop
```

```
.data
array:
    .word 8, 1, 5, 2, 7, 9, 6, 4, 3, 10
```

运行截图



实验分析

本实验使用 ARMv7 汇编语言实现了升序排序程序（sort2_armv7.s）。程序在数据段中定义了一个包含 10 个整数的数组（8,1,5,2,7,9,6,4,3,10），并通过寄存器保存数组首地址。

排序过程采用冒泡排序算法，通过两层循环实现数据的比较与交换。外层循环控制排序趟数，内层循环逐对比较相邻元素。当当前元素大于后一个元素时，通过 str 指令交换两者位置，使较小的元素逐渐移动到数组前端，从而实现升序排列。

程序运行结束后，在 Memory 视图中通过标签 array 定位数组，可以观察到数据依次为：

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

结果与预期升序排列一致，说明排序程序运行正确。实验表明 ARMv7 汇编语言可以通过寄存器操作、条件分支（如 cmp、ble）以及内存访问指令（ldr、str）实现排序算法。